社団法人　電子情報通信学会
THE INSTITUTE OF ELECTRONICS,
INFORMATION AND COMMUNICATION ENGINEERS

信学技報
TECHNICAL REPORT OF IEICE.
SS94－37 (1994－11)

自然言語処理研究向き文書エディタの試作について

堤　正勝[†]　荒木　健治[††]　宮永　喜一[†]　栃内　香次[†]

[†]北海道大学工学部　[††]北海学園大学工学部

北海道大学 大学院 工学研究科
〒０６０ 札幌市北区北１３条西８丁目
Tel (011)716-2111
E-mail:tutumi@hudk.hokudai.ac.jp

あらまし　本稿では自然言語処理研究向けの、ビジュアルな環境における単語管理機能を有する文書エディタの試作版について紹介している。本エディタの特長は、辞書を内部に持ち、単語文字列を管理し、ロードされる文書ファイルの単語が辞書へのポインタで表されることにある。この構成により、自然言語処理で頻繁に行なわれる、文字列のマッチング処理が省略され、効率がよいものになる。また内部のデータ構造の柔軟性により、単語のグループ化や、構造化、文章の構造解析の表現が容易である。さらに本エディタは表示に関して考慮しており各種編集モードに応じた表示をする。開発環境としては、本エディタ内部の各種ルーチン群がライブラリとして利用できるので、これを利用して応用モードを作成することで、容易にビジュアルな操作環境を構築できる。

和文キーワード　自然言語処理、日本語処理、文書エディタ、単語管理

Impremention of a Document Editor for Natural Language Processing.

Masakatsu TSUTSUMI[†] Kenji ARAKI[††] Yoshikazu MIYANAGA[†] Koji TOCHINAI[†]

[†]Faculty of Engineering, Hokkaido University
[††]Faculty of Engineering, Hokkai Gakuen University

N13, W8 Kita-ku
Sapporo 060, Japan
E-Mail:tutumi@hudk.hokudai.ac.jp

Abstract

In this paper, we describe on the implementation of a document editor for the research of natural language processing. It gives functions to manage words and and texts in a visual enviroment. This editor includes words-dictionaries and manages word-strings, and the words in loaded texts are expressed as the pointers to dictionary entries. Since this structure omits much of string pattern matchings, a high efficiency is achieved. By using a flexible data structure, it is easy to express the grouping and structuring of words, and also to express the result of the structure analysis of sentences. Besides them, this editor gives useful internal functions as library routines, it is easy to add applications for researches on the visual operational enviroment.

英文 key words　Natural Language Processing, Japanese Language Processing, Document Editor, Management of Words

1．まえがき

自然言語処理研究における計算機利用の形態としては、一般に計算機言語を使用して、文章を解析し目的の形式に変換し、必要ならば修正を加え、統計情報等を取得して、目的となる結果を出力するという流れが一般的である。この目的には、ＬＩＳＰやＰＲＯＬＯＧといった問題指向言語プロセッサが用いられる場合が多い。しかし、これらの言語は、小規模な記号処理等に対しては非常に有利な計算機環境を研究者に提供するという利点があるが、扱うデータの規模を大きくしにくいことや、提供される機能の範囲を越えた使用をするためのオーバヘッドが大きいという問題も存在する。とくに自然言語処理研究では、大量の語彙を用いて実験を行なう場合があり、この様な要求に応えるにはやはり汎用の手続き型言語、ＵＮＩＸではＣ言語を使用しなければならない。ところが、他の高級言語に比べ、Ｃ言語に備わる機能は、比較的細かい処理が記述できる半面、自然言語処理のための高度な機能は提供されていないのが現状である。このことについて、例えばＸウィンドウの様な高度な環境をライブラリ等の形で提供すれば、自然言語処理研究に非常に貢献できるのではないかと考えられる。これらは、具体的には単語管理や、検索、登録、パターンマッチング、さらには、表示や、操作のためのユーザインターフェースである。個々の機能は、それぞれＵＮＩＸのシステムコールや標準ライブラリから得られるものもあるが、本稿では、それらをひとつの統合環境として提供することを考え、実際にエディタとして試作し、研究用のアルゴリズムをこのエディタへのアプリケーション機能として追加し、簡単なキー操作でそのアルゴリズムを呼び出す形で、その結果をフルスクリーンエディタにより操作できる環境を構築した試作システムについて述べる。なお、本エディタは自然言語処理研究の各方面での応用が考えられるが、我々の当面の自然言語処理研究対象の具体例として、音声認識により得られる誤りを含む文字列を入力し、正しいと推定される候補を当てはめ、原文を復元するといった問題[1][3]を考えている。

2．本エディタの特長

2．1　通常のエディタとの違い

通常のエディタは、内部のデータ構造として、文字列のリスト、あるいは文字のリストを行として扱う行のリストをファイルとして扱う構造をとる。これに対し、本エディタは、自然言語処理の処理単位に適した単語単位で文書データを管理し、単語構造体を指すポインタのリストを行とした行のリストをファイルとして管理する。ここで、ファイルには単語にならない文字列が存在すると考えられるが、そのような文字列は未分割文字列として別に管理し、表示の段階で合成して通常のファイルとして表示する。

本エディタは辞書を起動時にロードし、辞書ファイルを単語構造体に展開する。単語構造体は、ファイルとは別に辞書として主記憶にロードする。この様にすることでファイル中に存在する全ての同一の単語は同一のポインタ値を持つので単語の同一性のチェックの毎に文字列比較を行なう負担を省く事ができる。また、単語構造体は、同音語を指すポインタで、リング状の構成になっているので、同音語の選択が容易にできる。つまり、単語確定というプロセスが存在しない。また、ファイル全体に対する一括処理による認識や解析により、同じ誤りが複数箇所で発生する場合に、それらを一括して修正する機能も有する。

2．2 既存エディタに倣う部分

本エディタは研究のための環境を提供することが目的なので、使い勝手のよさを必要とする。そこで、本質的に求めるもの以外は、画面構成など既存の便利なエディタに倣ったものにした。具体的にはモード概念を導入し、種々の情報を編集できるようにした。例えば、ファイル管理については、ディレクトリを表示して、削除、名前変更、移動、複写などの機能をより少ないキータッチで呼び出せるようにした。また例えば、辞書管理として、語彙や付属情報を編集できるモードを付加している。

2．3　単語管理

本エディタは自然言語処理研究を対象とし、形態素解析を必要とするさまざまな研究分野に利用しやすい辞書情報を扱うため、意味を中心とした単語管理を行なう。述語部は、意味の中心となる語幹を登録し[4]、変化部は叙述表現部として別の語とする。動詞は子音で区切ることにより活用行といった処理負担の増える情報を削減している。形容詞は、音便の場合が特殊になるため通常の学校文法の語幹で登録する。とくに形容詞については語尾パターンを拡張することにより品詞の転成を起こすので（例：「広い」＝＞「広める」）語尾調査をする予定である。なお、このような述語語尾に関しては、学校文法とは異なる定義をしているので、実際の文章にあたって、拾いだしを行なって、パターンの統計処理を行なうことにより叙述表現の単語としての決定をする。なお、これは現段階では、拾い出しの段階で報告するに至っていない。以上のようにすることで、できるだけ登録パターンを減らし、意味中心の単語管理を行なう。このようにして、単語の母数を減らし、メモリ効率の点でも向上をはかっている。

2．4　文書の保存

本エディタは形態素解析を入力時に行なうが、毎回文書を

ロードする度に同じ処理をするのは無駄である。そこで、「括弧形式」と我々が呼ぶ文書形式 [2] を定義して形態素解析の結果の情報を完全に保存する。「括弧形式」は日本語や中国語などの表意文字系言語を対象にしており、辞書や保存テキスト内では、以下のように表現している。

フォーマット：(読み／表記／属性［／その他の情報］..)

例：　(hana/花/n)

読みフィールド：ローマ字で表記する
表記フィールド：表示や印刷をするための文字列
属性フィールド：品詞など。
その他の情報　：頻度、履歴、誤り回数、連接情報、
翻訳情報（和英）など必要に応じて。

現在の版では英文等を扱う場合のフィールド構成：

フォーマット：(発音／単語／属性［／その他の情報］...)

なお、現段階では我々のエディタ使用目的を優先的に考えており、英文の管理の詳細については保留している。

形態素解析を行なった文書中の単語は上記括弧形式で表現して、単語の列として８０桁に収めてセーブする。なお、このままでは、通常のテキストとはちがい、読みにくいので、表記のみを取り出すツールも用意した。

３．エディタの内部構成

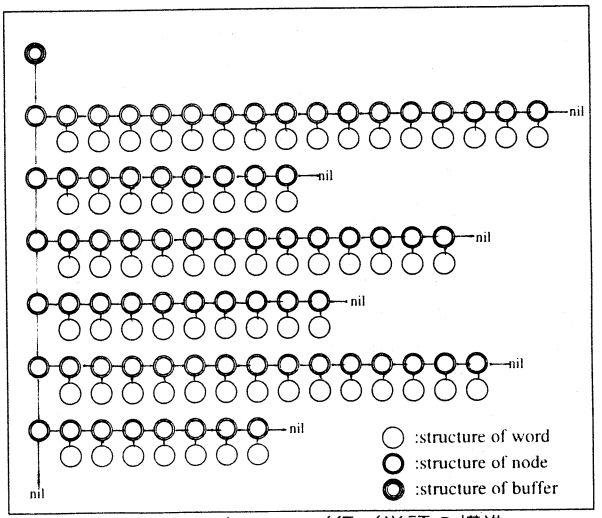


図３．１　バッファ／行／単語の構造

本エディタはノード構造体を用いて種々の情報をリストや木構造で管理している。

３．１　マルチバッファエディタ

本エディタは辞書をロードするので、プロセスあたりひとつのファイルを扱っていると、計算機の利用効率が悪くなるので、複数のバッファを用意して、幾つものファイルを扱うことができる。このための構造として、エディタの内部構造はまず、バッファのリストからなる。

３．２　バッファについて

バッファをリスト構造で扱い、バッファにモードという概念を付加して種々の情報を管理することは先に述べた。モードを用意することにより、各モードの特殊な機能が複雑なキーストロークによる呼び出しにならず、編集作業の効率が良くなると考えられる。また、モード毎に表示形態を変えることができるので編集モード毎に最も適した画面構成を作成できる。このことは、テキストファイルのような行のリストを主構造としてもつ簡単なテキストの表示や、二分木検索を行なう複雑な構造の辞書を表示するなど、柔軟な対応が可能である。モードはバッファ毎にセットされ、カレントバッファを切替えることにより、呼び出される表示関数も自動的に切り替わり、エディタ全体の構造が見通しのよいものになっている。

３．３　モードについて

ここでは、モードの定義について説明する。モードは、個々のバッファが開設される以前に初期化され、その構造は、モード名と４つの関数、入力変換、終了処理、表示管理、キーボード入力からなる。以下に個々の関数の果たすべき役割を説明する。新たなアプリケーションモードを追加するにはこのルールを守って作成する。

３．３．１　入力変換

入力変換は、バッファが開設され、モードが割り当てられた直後にバッファ管理から呼び出される機能で、ファイルのロード、内部形式への変換、表示の準備などを行なう。内部形式への変換はたとえば、辞書なら、辞書要素を構造体へ変換して、検索を容易にするため木構造に展開する処理を行ない、文書ファイルなら、形態素解析を行ない、行、単語列の構造に展開する。表示の準備としては、操作状態の初期化、バッファのカーソル位置情報の設定などを行なう。ファイルのタイムスタンプをバッファに記録し、編集操作ごとに更新している。こうすることで、同一ファイルを二重にロードした場合の調停をとるようにしている。また、ファイルのアクセスモードもバッファに記録し、編集可能／不可能を制御している。現在のところ未対応であるが、ＵＮＩＸの file コマンドのように filetype を判定し、誤ってバイナリファイルを読み込まないようにしたり、異なるコード系を使用したファイルのロードも可能にする予定である。

3．3．2　終了処理

終了処理は、エディタ終了時、あるいは、キーボード入力によりバッファ削除の要求があった時、バッファの編集フラグがセットされている場合、バッファ管理から、バッファ削除直前に呼び出される関数である。ここでの処理は主に編集されたテキストや、辞書のセーブを行なう。辞書は、安全のため、世代管理で、バックアップを数世代保存している。本エディタはバックアップファイル名は「ファイル名.999」の形で0から順に作成しており、セーブの度に旧ファイルの番号を全て＋1し、最新のファイルは常にロードファイル名と同一になるようにしている。本エディタは辞書を削除しないので、ＵＮＩＸの機能crontab で、定期的に不要になった番号の大きなの辞書を削除する。

3．3．3　キーボード入力処理

kterm は ioctl により raw モードに設定して使用しているので、1文字単位の入出力が行なえる。キーボードからの文字取得は1箇所で行ない、ここがメインのループになっている。文字が取得されると、カレントバッファのキーボード入力関数が呼ばれる。キーボード入力関数では、操作状態を示すサブモードにより分岐することで、マルチストロークのキーバインドに対応している。キー文字への機能の割り当ては、サブモード毎に switch-case 文で文字を判定して関数を呼び出すかたちで行なっている。基本モードのキーボード処理は汎用に作成しており、アプリケーションモードのキーボード入力処理からデフォルト処理として呼び出すことができる。

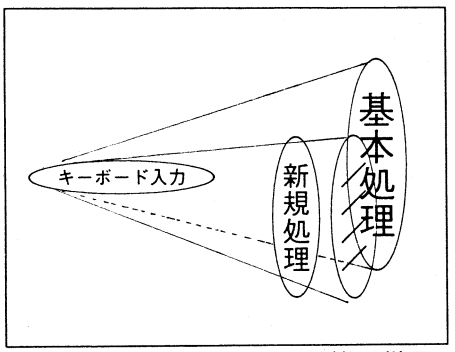


図3．2　キーボード定義の継承

3．3．4　表示管理

表示管理はモード毎の多彩な表示形態に対応するために、モードを構成する要素となっている。カレントバッファを切替えることにより、バッファを引数に呼び出される表示管理も切り替わり、表示内容全体が描画される。表示管理はキーボード入力同様、kterm を ioctl により raw モードの状態で使用している。画面構成は、表示可能なキャラクタマップのサイズにできるだけ多くの情報量を得るため、カラー、アンダーライン、ボールド、反転など、端末のバリエーションをフルに利用している。このため、白黒端末での使用は不可能に近い。表示管理の呼び出しは、2系統あり、キーボード入力管理から呼び出された関数により、表示状態に変化があるときに行なわれる場合と、Ｘウィンドウシステムにおいて一般に行なわれるウィンドウのリサイズによる割り込み処理から呼び出される。ウィンドウのサイズが可変であることは、表示管理を複雑にしているが、この問題に対処しないで８０×２５を固定にすると、表示が乱れたり、カーソル位置の制御にも問題を及ぼすのでこれをきちんと管理する必要がある。

4．テキストファイル

ファイルの読み込みは上述のとおり、名前とモードをを与えられたバッファが開設された直後、初期化処理で行なわれる。テキストファイルはテキストモードで開設されたバッファに読み込まれ、形態素解析済みの括弧形式ファイルか、通常の文書であるか判定され、通常の文書ならば、形態素解析が行なわれ、括弧形式テキストならば、所定の字句処理が行なわれ、それぞれ内部表現に展開される。形態素アルゴリズムは、現在のところ、最低限の処理として最長一致で行なうが、当研究室でかつてから報告している多段階分割法をインプリメントする予定である。テキストファイルの内部表現は、形態素に関しては単語辞書へのポインタをリスト構造で管理し、これを行として、更に行をリスト構造に構成する。形態素解析に失敗し、未分割文字列となった部分は、そのまま文字列ノードとして管理され、単語ノードとは区別している。テキストファイルは先に述べた「括弧形式」でセーブされるほか、コマンドにより通常のテキストとしてもセーブできる。これは形態素解析のコストを考慮しての措置である。

なお、現在の版は試作版であり、新規文書の編集も可能であるが、最低限の機能だけに限っているので、今後、機能の充実を図って対応する予定である。とくに本エディタの構造はすべてノードの連鎖で管理されておりノードに種類はあるが、どんなノードも互いに接続可能なので、バッファノードをテキストファイルのノードの中に混在させることができ、ハイパーテキストが構築できるといった利点がある。

5．辞書管理モジュール

辞書も編集可能バッファとして管理されることは先にも述べている。ここではその構造について説明する。

5．1　辞書構造

辞書は検索アルゴリズムに適する構造であるために、テキ

ストとは異なるバッファ形態をとる。そこで検索手法を基準に構造を決定する。本エディタは辞書を編集する機能があり、項目の追加、削除が起こるので、検索以前にソーティングを必要とするアルゴリズムは採用しにくい。そこで木構造を採用した。木構造は登録時に最低限の負荷でソーティングがなされるので、この問題に都合がよい。

5．2　木構造辞書の改善

木構造による二分木検索には重大な欠点がある。一般に辞書ファイルは人間が見るためにはソーティングされていると都合がよい。これに対して、ソーティングされた辞書を二分木に展開しようとして、順番に登録してゆくと木構造としては甚だ都合のわるい、片方の枝だけのびてしまい、検索時にはリニアサーチと同じになる。このような不都合を解消するために二つの手法を考案した。

5．3　二分法による木構造登録。

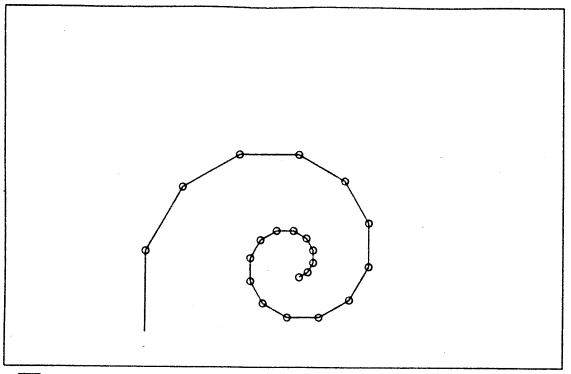


図5．1　ソートされた辞書を登録した時

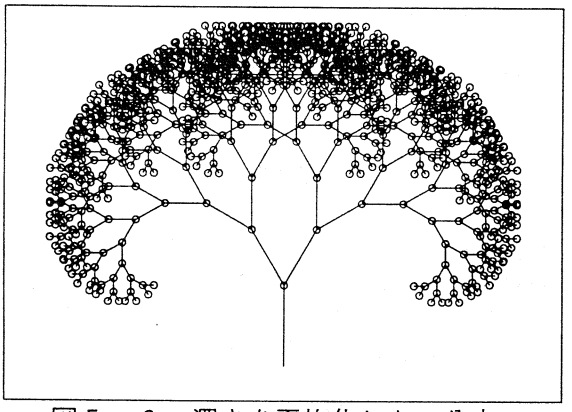


図5．2　深さを平均化した二分木

まず、ソーティングされた辞書を用意する。最初に中央の単語を登録する。然る後、前の半分と後の半分の部分列についてそれぞれ同様の登録処理を再帰的に行なう。こうすることで、深さの平均化された木が構築できる。実際のインプリメントにおいては、エディタの中にこのためのワークメモリを作成するとガベージとなるので、子プロセスを呼び出してこの処理をさせている。現在の版ではこの手法を用いている。

5．4　学習機能を有する二分木構造辞書[5]

木の深さを減らす手法としては上記と同様であるが、深さの均一化よりも、使用頻度に着目して使用頻度の高い語ほど、ルートに近くなるように、参照毎に枝の付け変えを行なう手法である。今後、インプリメントする予定であるがすでにこの手法の有効性は確認済みである。

5．5　ハッシュ法による検索

大語彙を扱う場合には検索がネックになる場合がある。このために、主記憶の許す限りメモリを使用する手法が必要になる場合も考えられる。そこで、ハッシュ法のアルゴリズムを使用可能にしておく必要がある。具体的には、アルゴリズムをリンクしておき、ハッシュ法による検索が初めて呼び出された時に、テーブルの作成を行なうようにする予定である。

5．6　単語の同音語リング

木構造のノードは単語構造体を指している。また、単語構造体は、同音語を指している。最後の単語は最初の単語を指している。この構造にしておくと同音語選択のときに都合が良い。

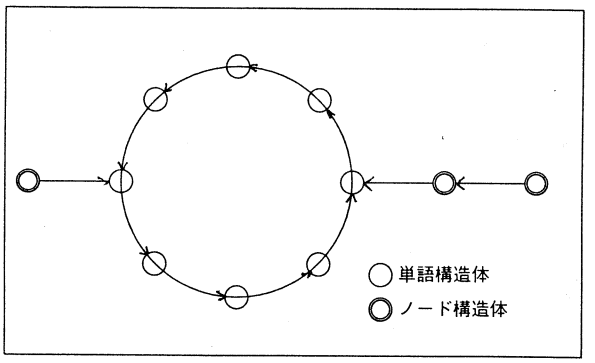


図5．3　同音語管理

5．7　二重の検索木

上記は辞書の主構造である。辞書は読みによる検索木で同音語リングを指しているように表現したが、これだけでは、かな漢字変換は可能だが、形態素解析には不適当で、表記による検索が必要である。そこで、表記による検索木も作成し、二重の検索木としてある。同音語と同様に、同表記リングを指している。

5．8　辞書の種類

上記の辞書は本エディタの主辞書であり、起動時にロードされる。テキストの単語ノードはすべてこの辞書の単語をポインタで参照する。しかし、日本語に存在するすべての語を網羅したものではないので、追加登録が必要になる。その登録作業を補助するための辞書を用意している。

5．8．1　単漢字辞書

単漢字辞書はＪＩＳ第１、第２水準のすべての漢字に関する辞書であり、二つ用意してある。一つは部首による検索を行なう辞書で部首名をキーに検索するもの。部首名を知っている場合に有効である。もう一つは、読みによる検索を行なう辞書で、音読み、訓読みが区別されている。これらの辞書を用意することにより、ＪＩＳコードのすべての漢字が、ワープロ辞書なしで、入力できる。主辞書とはアクセスの仕方が違うこと、また、かな漢字変換における不都合のため、異なる辞書モードを割り当てている。検索速度にそれほど強い要求はないので、リニアサーチで済ませてある。単語情報は文字列のまま管理している。

5．8．2　既存辞書

著者らは、以前からかな漢字変換、べた書き変換などで、使用していた辞書がある。また、フリーウェアでネットワークから取得できる辞書もある。こういった辞書にはそれぞれのフォーマットがあるので初期化部、あるいは、参照する関数で変換する必要がある。それぞれモードを設定し、バッファで管理している。

6．基本モードについて

本章では基本モードとして用意しておかねばならない幾つかのモードについて述べる。なお、モードはモジュール単位で管理している。

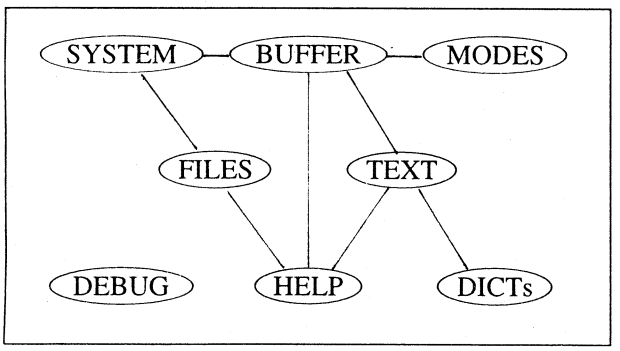


図６．１　管理モジュール構成

6．1　バッファ管理モジュール

すべての構造あるデータを管理するモジュール。既存バッファの検索、表示切替え、初期化、終了処理の呼び出しを行なう。バッファ管理モードをもつ。バッファ管理モードでは、画面にバッファの一覧を表示してカレントバッファを選択させる。バッファ一覧にはバッファの属性、ファイル名などを表示する。バッファの中にはファイルとして存在しないものもある。

6．2　システム管理モジュール

スターとアップファイルの読み込みを行ない、各モードの環境情報を提供するシステムモードを管理する。スターとアップファイルは通常の可読ファイルあるいは、実行可能属性のあるシェルスクリプトで記述される。本モードの初期化部では、ファイルの属性により判断し、ファイルそのものを読み込むか、シェルスクリプトの実行結果を読みとるかする。シェルスクリプトの形式は、ネットワーク環境での複数のホストや端末を識別する必要のあるモジュールで変数が必要な場合に有効である。なお、実行属性がついていれば、バイナリファイルでもよい。本モードはリードオンリーの、システムバッファとなり、セーブは起こらない。

6．3　ファイル管理モジュール

ディレクトリエントリ一覧を表示し、編集するファイルの選択、削除、複写、移動、コマンド実行などを少ないキーストロークで呼び出す。本エディタを、パラメタなしで立ち上げるとこのモードになる。

6．4　ヘルプモジュール

各モードにおけるマニュアルを表示する、最も単純なモード。表示管理は、テキストモードを流用している。

6．5　テキスト管理モジュール

文書管理のデフォルトとなるモジュール。ここから、形態素解析や、かな漢字変換などの機能が呼び出される。また、行ノードから表示行を作成したり、それらを組み合わせて画面を作成したりするための、ルーチンを提供している。

6．6　モード管理モジュール

モードのリストを管理するモジュール。何もしない、ただのリストである。モードの検索はバッファ管理が行なう。

6．7　デバッグ支援モジュール

エディタの開発では標準出力に対する printf を用いてデバッグすることは表示を乱すので使用できない。そこでこのバッファに出力して内容を確認する。

7．インプリメント

本エディタは、ホストマシン Sparc Station 10, 主記憶 64M bytes に構築し、カラーX端末上で、kterm を起動した上で動作する。漢字フォントは16×16ドットあるいは24×24ドットのものを使用している。プログラムは現在のところC言語で約10000行程度である。漢字コードはEUCを用いている。画面構成は、emacs ライクに作成した。(図：表示例)。

コードは kterm 上を仮定して termcap や curses 等のライブラリは使用せず、直接エスケープシーケンスで記述して制御している。

起動は、ファイル名を付加して立ち上げると、ファイル編集からはじまり、なにも指定しないと、現在のディレクトリに対するファイル管理モードになるように設定してある。

8．おわりに

本報告では、自然言語処理研究のために必要な単語管理を機能として有するエディタの試作版について報告した。本エディタは、内部の辞書とテキストを編集するために必要な、表示管理やキー入力の管理をライブラリとして利用できる形で作成しているので、新たなモードを定義し、C言語によるモジュールとして作成し、エディタにリンクすることにより、任意のアルゴリズムを容易に追加できるようになっている。現在、文書編集機能、辞書編集機能、ヘルプ表示、ファイル管理機能、かな漢字変換入力など、基本部分を作成したところである。現在は kterm 上で動作するが、Xウィンドウ版も手掛けておりGUI対応にも拡張可能である。

今後の予定としては、実際に文書を編集しながらこれらの機能をより充実させること、また、日本語処理、とくに音声認識結果に対する誤りの修正の問題に応用してゆきたい。また、ネットワークのニュースから文書を自動的に取得して、未知語を取得してゆき、辞書の充実をはかる方法も実現したい。

参考文献

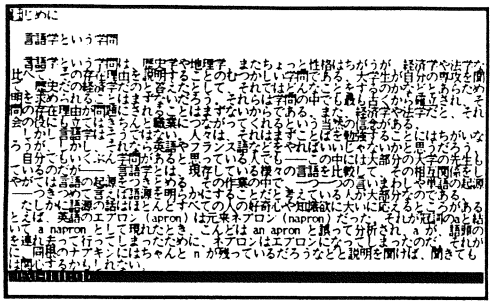
[1] 荒木: 多段階分割復元法による日本語文書処理に関する研究：北海道大学工学部，学位論文，昭和63年

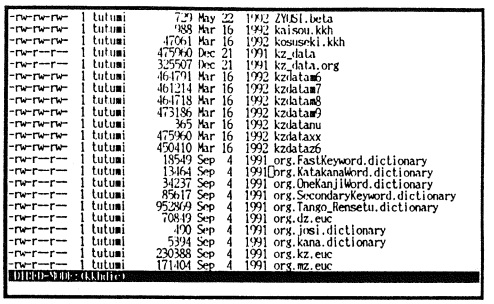
[2] 堤　正勝、荒木健治、宮永喜一、栃内香次：多段階分割復元法を用いた連続音声認識の誤りの復元：電子情報通信学会，音声研究会資料，SP89-92 (1989).

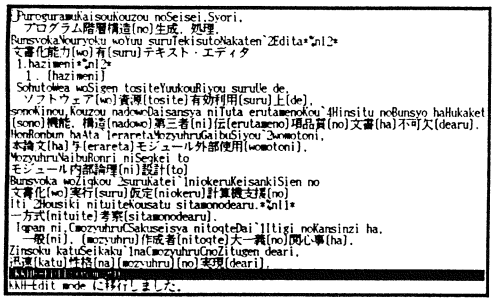
[3] 堤　正勝、荒木健治、宮永喜一、栃内香次：自然言語処理における実験用文書データの取得及び保存について：平成4年度電気関係学会北海道支部連合大会，北見 (1992).

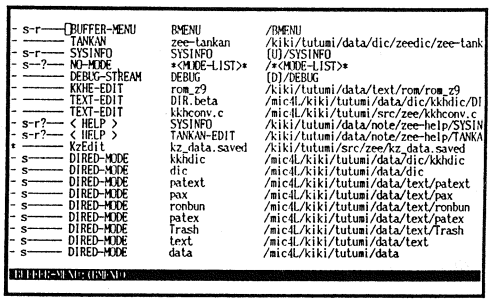
[4] 久光　徹、新田義彦：漢字かな混じり文形態素解析における非サ変動詞の分割単位設定について：情報処理学会論文誌，Vol.30，No.2，pp.159-167(1991).

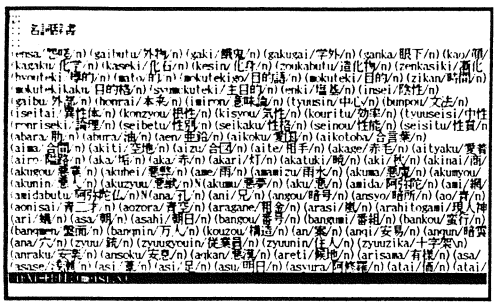
[5] 堤 正勝：学習機能を有する木構造辞書について：北海道大学工学部，卒業論文，昭和63年


はじめに
言語学という学問
言語学というと学問は、歴史学や地理学、またちょっと性格はちがうが、経済学や法学などに比べて、その存在理由を説明することのむつかしい学問である。大学生が自分の専攻を聞かれ、「言語学だの経済学だのと答えたとして、それではどんなことをするのかなどと聞かれるため、曲を求められることは予想されるであろう。それらは学問の中でも最も古くから存在され、その存在理由が問題にされることはまずないからである。また、経済学や法学だと、それ会の役にも立てばきちんと職業につながってくれるという当然の答えがある。
しかし言語学はそうではない。人々は、それはまずことばを勉強することにはちがいないろうが、「しかし、それから英語やフランス語などをやればいいじゃないかと思うだろう。
自分でもいくらかん若干があると思っている人でも一この中には大部分の大学の先生もいるのだが一言語学とは、現存している様々の言語を比較して、その相互関係をしやがては言語の起源をさぐろうとする、その作業の中で、ことばの正しまわしや単語の起源一つさぐって言語を明らかにすることだと考えている人が大部分なのである。
たしかに言語の話には8割ほどとんどすべての人の好奇心や知識欲に大いに応えるところがあるとえば、英語のエプロン（apron）は元来ネプロン（napron）だった、それが冠詞のaと結いて、a napronとして現れたとき、こんどはan apronと誤って分析され、aが、語頭のを通り越して行ってしまったために、オプロンはエプロンになってしまったのだ。それがに、同様のナプキンにはちゃんとnが残っているだろうなどと説明を聞けば、聞きでもは関心するかもしれない。
[KZ-EDIT:[1]]


-rw-rw-rw- 1 tutumi 721 May 22 1992 ZYUSI.beta
-rw-rw-rw- 1 tutumi 988 Mar 16 1992 kaisou.kkh
-rw-rw-rw- 1 tutumi 47061 Mar 16 1992 koususkl.kkh
-rw-r--r-- 1 tutumi 475960 Dec 21 1991 kz_data
-rw-r--r-- 1 tutumi 325507 Dec 21 1991 kz_data.org
-rw-rw-rw- 1 tutumi 464791 Mar 16 1992 kzdata6
-rw-rw-rw- 1 tutumi 461214 Mar 16 1992 kzdata7
-rw-rw-rw- 1 tutumi 464718 Mar 16 1992 kzdata8
-rw-rw-rw- 1 tutumi 473186 Mar 16 1992 kzdata9
-rw-rw-rw- 1 tutumi 365 Mar 16 1992 kzdataau
-rw-rw-rw- 1 tutumi 475960 Mar 16 1992 kzdataxx
-rw-rw-rw- 1 tutumi 450410 Mar 16 1992 kzdataz6
-rw-r--r-- 1 tutumi 18549 Sep 4 1991 org.FastKeyword.dictionary
-rw-r--r-- 1 tutumi 13464 Sep 4 1991 org.KatakanaWord.dictionary
-rw-r--r-- 1 tutumi 34237 Sep 4 1991 org.OneKanjiWord.dictionary
-rw-r--r-- 1 tutumi 85617 Sep 4 1991 org.SecondaryKeyword.dictionary
-rw-r--r-- 1 tutumi 952869 Sep 4 1991 org.Tango_Rensetu.dictionary
-rw-r--r-- 1 tutumi 70849 Sep 4 1991 org.dz.euc
-rw-r--r-- 1 tutumi 400 Sep 4 1991 org.josi.dictionary
-rw-r--r-- 1 tutumi 5394 Sep 4 1991 org.kana.dictionary
-rw-r--r-- 1 tutumi 230388 Sep 4 1991 org.kz.euc
-rw-r--r-- 1 tutumi 171404 Sep 4 1991 org.mz.euc
DIRED-MODE: (kkhdic)


Puroguramukaisoukouzou noSeisei.Syori.
プログラム階層構造(no)生成．処理．
Bunsyoka¥kouryoku wo¥yu suru¥tekisuto¥Nakaten`2Edita*¤nl2*
文書化能力(wo)有(suru)テキスト・エディタ
1.hazimeni*¤nl2*
1. [hazimeni]
Sohutoweа wo¥igen tosite¥yuukou¥iyou surule de.
ソフトウェア(we)資源(tosite)有効利用(suru)上(de).
sonoKinou,kouzou nadowoDaisansya niTuta erutamenokou`4Hinsitu noBunsyo haHukaket(sono)機能，構造(nadowo)第三者(ni)伝(erutaseno)項品質(no)文書(ha)不可欠(dearu).
HonRonbun haAta [erareta¤zyuhruGaibuSiyou`3womotoni,
本論文(ha)与(erareta)モジュール外部使用(womotoni),
¤zyuhruNaibuNori niSeikei to
モジュール内部論理(ni)設計(to)
Bunsyoka wo¥gikou `2surukatei`lniokeruKeisankiSien no
文書化(wo)実行(suru)仮定(niokeru)計算機支援(no)
Iti `2Housiki nituite¥kousatu sitamonodearu.*¤nl*
一方式(nituite)考察(sitamonodearu).
Ingan ni,Cmozyuhru¥Sakusseisya nitoqte¤Dai`1Itigi noKansinzi ha.
一般(ni)，(mozyuhru)作成者(nitoqte)大一義(no)関心事(ha).
Zinsoku katuSeikaku`lnaCmozyuhruGnoZitugen deari.
迅速(katu)性格(na)(mozyuhru)(no)実現(deari).
KKH-edit mode に移行しました．


- s-r--- BUFFER-MENU BMENU /BMENU
- ------ TANKAN zee-tankan /kiki/tutumi/data/dic/zeedic/zee-tank
- s-r--- SYSINFO SYSINFO [U]/SYSINFO
- s--?--- NO-MODE *<MODE-LIST>* /*<MODE-LIST>*
- ------ DEBUG-STREAM DEBUG [D]/DEBUG
- ------ KKHE-EDIT rom_g9 /kiki/tutumi/data/text/rom/rom_g9
- ------ TEXT-EDIT DIR.beta /mic4L/kiki/tutumi/data/dic/kkhdic/DI
- ------ TEXT-EDIT kkhconv.c /mic4L/kiki/tutumi/src/zee/kkhconv.c
- s-r?--- < HELP > SYSINFO /kiki/tutumi/data/note/zee-help/SYSIN
- s-r?--- < HELP > TANKAN-EDIT /kiki/tutumi/data/note/zee-help/TANKA
* ------ KzEdit kz_data.saved /kiki/tutumi/src/zee/kz_data.saved
- s------ DIRED-MODE kkhdic /mic4L/kiki/tutumi/data/dic/kkhdic
- s------ DIRED-MODE dic /mic4L/kiki/tutumi/data/dic
- s------ DIRED-MODE patext /mic4L/kiki/tutumi/data/text/patext
- s------ DIRED-MODE pax /mic4L/kiki/tutumi/data/text/pax
- s------ DIRED-MODE ronbun /mic4L/kiki/tutumi/data/text/ronbun
- s------ DIRED-MODE patex /mic4L/kiki/tutumi/data/text/patex
- s------ DIRED-MODE Trash /mic4L/kiki/tutumi/data/text/Trash
- s------ DIRED-MODE text /mic4L/kiki/tutumi/data/text
- s------ DIRED-MODE data /mic4L/kiki/tutumi/data
BUFFER-MENU: (BMENU)


名詞辞書
:
:
: